

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет информатики, математики и компьютерных наук
Власов Артём Дмитриевич

**Разработка высокопроизводительного парсера для извлечения
адресной информации из русских текстов**
КУРСОВАЯ РАБОТА
по направлению подготовки 09.03.04 «Программная инженерия»
образовательная программа
«Компьютерные науки и технологии»

Научный руководитель
к.ф.-м.н., доцент базовой кафедры
информационных систем и
технологий
Улитин Б. И.

Соруководитель
Главный архитектор центра
разработки ООО «Медиа-Майн»
Раткин К.В.

Нижний Новгород 2026

Аннотации

В данной работе рассматривается задача повышения производительности синтаксического разбора русскоязычных почтовых адресов в системах извлечения именованных сущностей. Выполнен сравнительный анализ существующих подходов извлечения сущностей из текстов на русском языке, представленных в библиотеке Yargy. В рамках вычислительного эксперимента продемонстрировано, что предложенная Rust-реализация библиотеки Yargy позволяет существенно увеличить скорость обработки адресов за счёт высокой производительности языка Rust и совокупностью различных методов оптимизаций. Полученные результаты демонстрируют перспективность применения языка Rust для построения высокопроизводительных систем для обработки адресной информации из русскоязычных текстов в промышленных условиях. Практическая значимость работы заключается в создании высокопроизводительной Rust библиотеки RENERT, ориентированной на извлечении информации из текстов на русском языке для тех областей, где критически важны высокие точность и производительность. Исходный код и документация библиотеки RENERT доступны по следующим адресам: [Github](#), [Документация](#).

Ключевые слова: извлечение адресов, русскоязычные тексты, NER, алгоритм Эрли, контекстно-свободная грамматика, Yargy, RENERT

Оглавление

1. Введение	4
2. Аналитический обзор предметной области	6
2.1. Задача извлечения адресной информации	6
2.2. Особенности русских адресов	7
2.3. Методы решения задачи извлечения именованных сущностей ...	10
2.4. Грамматические подходы и библиотека Yargy	11
3. Теоретическая основа и архитектура решения	14
3.1. Контекстно-свободная грамматика	14
3.2. Алгоритм Эрли	16
3.3. Архитектура библиотек Yargy и RENERT	18
4. Практическая реализация и оптимизация	21
4.1. Выбор Rust	21
4.2. Программная реализация	23
4.3. Оптимизация в библиотеке RENERT	27
5. Вычислительный эксперимент	31
5.1 Описание эксперимента	31
5.2. Результаты эксперимента	32
5.3. Анализ полученных результатов	32
6. Заключение	34
Список литературы	36

1. Введение

В современных промышленных информационных системах одной из ключевых задач обработки текстовых данных является извлечение именованных сущностей (Named Entity Recognition, NER). В пользовательских данных, логистических, накладных и банковских документах регулярно встречаются такие сущности, как фамилии и имена, даты, адреса электронной почты, домены, номера телефонов и почтовые адреса. Следовательно, выделение и структурирование таких данных является важной частью автоматизации документооборота, логистики, CRM-систем и государственных информационных сервисов.

Эффективность алгоритмов извлечения именованных сущностей в значительной степени зависит от языка обрабатываемого текста. Для английского языка эта задача была тщательно изучена, и для достижения высокой точности и высокой скорости обработки существует большое количество готовых решений — библиотек, статистических и нейросетевых моделей. Однако для русского языка подобных решений значительно меньше. Это связано как меньшим количеством специализированных наборов данных, так и сложностью самой языковой системы.

Для русского языка характерна развитая морфологическая система, в которой слова могут изменяться по падежу, числу, роду и времени. Из-за этого в русских текстах одна и та же сущность может встречаться в большом количестве словоформ, что создает дополнительные трудности в задаче извлечения именованных сущностей.

Особенно заметна данная проблема при обработке почтовых адресов. Адреса относятся к числу одной из наиболее распространённых сущностей в системах обработки текстовой информации, однако их структура часто оказывается неполной или неоднородной. Несмотря на существование формальных правил записи адресов в Российской Федерации, на практике

используются сокращения, различные варианты написания, пропуски отдельных элементов и нестандартные обозначения.

Одной из наиболее известных библиотек для извлечения именованных сущностей из русских текстов является библиотека Yargy [1]. В основе работы библиотеки лежит контекстно-свободная грамматика и алгоритм Эрли [2] для синтаксического анализа, чтобы проводить разбор текстов по заранее собранным правилам.

В тоже время производительность библиотекой остаётся низкой. Реализация на языке Python обрабатывает в среднем около 15 адресов в секунду, что ограничивает применение Yargy в высоконагруженных системах для обработки больших объёмов данных. Следовательно, значение скорости обработки текстов обретает критическое значение для систем с высокой нагрузкой и большим массивом данных.

Таким образом, задача ускорения открытой библиотеки Yargy является актуальной и практически значимой для российских разработчиков программного обеспечения. Сочетание высокой точности грамматического подхода и высокой производительности имеет решающее значение для промышленного применения систем извлечения адресной информации.

2. Аналитический обзор предметной области

2.1. Задача извлечения адресной информации

Извлечение информации (Information Extraction, IE) представляет собой направление обработки естественного языка, ориентированного на автоматическое выделение из неструктурированных текстов большой размерности с последующим приведением их к структурированному виду. В отличие от поиска по ключевым словам, методы извлечения информации предлагают более глубокий анализ текста, включая выявление сущностей, связей между ними, а также события и их характеристики [3].

Одной из подзадач извлечения информации является извлечение именованных сущностей (Named Entity Recognition, NER). Именованные сущности представляют собой фрагменты текста, соответствующие определённым категориям объектов реального мира. Наиболее распространёнными сущностями считаются имена людей, названия организаций и географические объекты. Например, в предложении «Сергей переехал в Москву» можно извлечь такие сущности, как «Персона» и «Город».

По мере развития методов обработки текстов количество используемых категорий сущностей увеличилось. Помимо классических типов сущностей, современные системы способны выделять временные выражения, величины, процентные значения и другие типы данных из текстов. Также набор сущностей может расширяться при работе с определённой предметной областью: например, в медицинских текстах могут выделяться названия заболеваний, лекарственных препаратов и методов лечения.

Задача определения и извлечения именованных сущностей заключается в том, чтобы определить границы в тексте и классифицировать их по типу. При этом возникают дополнительные трудности, связанные с уникальностью.

Во-первых, не всегда очевидно, какие слова входят в состав сущности, особенно если она состоит из нескольких компонентов. Это приводит к проблеме выделения вложенных сущностей, когда один фрагмент текста может одновременно относиться к другому типу. Например, название организации может содержать имя человека. Во-вторых, возникает проблема неоднозначности единственных сущностей. Одно и то же слово или выражение может относиться к разным типам объектов в зависимости от контекста. Например, название может обозначать как географический объект, так и имя человека или организации. Для разрешения подобных неоднозначностей использовались методы, наблюдающие контекст окружения существ [3].

В рамках данной работы рассматривается частный, но практически значимый случай задачи извлечения информации — извлечение и структурирование почтовых адресов. Адреса представляют собой сложные составные сущности, включающие несколько компонентов (регион, город, улица, дом и др.), и характеризуются высокой вариативностью.

2.2. Особенности русских адресов

В Российской Федерации порядок записи почтовых адресов регламентируется нормативными документами, в частности правилами оказания услуг почтовой связи, утверждёнными приказом Минцифры России №382 [4]. Данные правила задают рекомендуемую структуру адреса и последовательность его компонентов — от более общих к более частным (страна, субъект Федерации, населённый пункт, улица, дом и др.).

Однако указанные требования носят в значительной степени регламентирующий и рекомендательный характер и ориентированы в основном на оформление почтовых отправлений. В реальных текстовых данных адреса записываются с отклонениями от установленного формата: допускаются перестановки компонентов, сокращения, пропуски элементов, а

также использование неформализованных обозначений. В результате наблюдается высокая вариативность представления адресной информации, что существенно усложняет задачу её автоматического извлечения и структурирования.

В ходе анализа используемого набора данных позволяет выделить ряд характерных особенностей русскоязычных адресов, которые необходимо учитывать при разборе текстов.

1. Вариативность структуры и неполнота данных

Адреса могут содержать как полный набор компонентов, так только их часть. Например, наряду с полными адресами вида «Россия, обл. Свердловская, г. Екатеринбург, ул. Гаршина, д. 3/3» также встречаются сокращённые записи, содержащие только регион и город: «Россия, край Краснодарский, г. Краснодар». Такая вариативность требует от алгоритмов обработки естественного языка поддержки обработки неполных адресных структур.

2. Отсутствие фиксированного порядка компонентов

Несмотря на нормативные рекомендации, в текстах порядок элементов может варьироваться. Например, район может указываться как до города, так и после него: «р-н Слободской, г. Слободской» или «г. Слободской, р-н Слободской». Такая вариативность затрудняет применение простых шаблонных методов.

3. Частое использование сокращений

В адресах активно применяется сокращённые формы обозначений административных единиц и объектов: «обл.» (область), «респ.» (республика), «г.» или «гор.» (город), «р-н» (район), «ул.» (улица), «пр-т» (проспект), «мкр.» (микрорайон), «д.» (дом), «к.» или «кор.» (корпус), «стр.» (строение), «зд.» (здание). При этом в рамках одного

приложения одни и те же сущности могут встречаться как в сокращённой, так и в полной форме, что требует дополнительной нормализации.

4. Сложные и составные наименования

Некоторые компоненты адреса могут иметь составную структуру и включать альтернативные названия или уточнения, например: «Ханты-Мансийский автономный округ — Югра», «Чувашская Республика-Чувашия». Такая структура адресов усложняет разбиение адреса на части.

5. Составные номера зданий

Нередко номера домов имеют сложный формат с уточнением квартиры, строения в здании с использованием разделителей: «д32/стр. 2», «д. 109/6/кв. 3», «дом 19/1/к.2». Такой формат требует отдельной обработки для корректного выделения компонентов.

6. Наличие нетипичных адресных объектов

В наборе данных среди адресов встречаются элементы, не относящиеся к классической схеме адресов «улица — дом», такие как «тер Самотлорское месторождение нефти» или «мкр. Центральный».

7. Использование падежей

Одни и те же элементы адреса могут встречаться в разных формах: например, «Свердловская область» и «Свердловской области», «улица Ленина» и «улице Ленина». Из-за этого при разборе текста необходимо учитывать падежные формы и другие грамматические особенности слов.

2.3. Методы решения задачи извлечения именованных сущностей

Современные подходы извлечения именованных сущностей из текстов можно разделить на 2 основные группы: методы, основанные на правилах (rule-based методы), и методы машинного обучения [3].

Методы, основанные на правилах. Представленный подход подразумевает использование заранее заданных правил, словарей предметной области или шаблонов для поиска сущностей из текстов. В качестве правил могут использоваться грамматики, синтаксические конструкции или регулярные выражения. Тем не менее, подобные методы требуют ручного формирования таких правил и значительных трудозатрат на этапе разработки, поскольку правила должны учитывать особенности конкретной предметной области.

Главными преимуществами rule-based методов являются высокая интерпретируемость за счёт длительной разработки и высокая обобщающей способности [3]. Каждое полученное решение может быть объяснено набором определённых правил, а детерминированность алгоритмов обеспечивает высокую предсказуемость, повторяемость и стабильность результата, исключая случайные ошибки или шумы.

Методы машинного обучения. Эти методы основаны на применении статистических и нейросетевых моделей для автоматического выделения сущностей из текстов. Их можно разделить на методы с ручным извлечением признаков (hand-crafted features) и end-to-end модели (модели полного цикла) [3].

Первая группа методов использует подготовленные лингвистические элементы: части речи, синтаксические особенности, словарные элементы и внешние базовые знания. Такие модели позволяют использовать сложный

язык, однако требуют предварительной обработки текста и настройки параметров пространства.

End-to-end модели работают непосредственно с текстом, используя векторное представление слов и предобученные языковые модели. Для русского языка особое значение имеют конструкции, наблюдающие морфологические особенности и работающие не только на уровне слов, но и на уровне символов. Кроме этого, на практике распространены комбинированные модели, объединяющие нейронные сети и вероятностные методы, например Bi-LSTM и CRF [3].

В последние годы для решения задач извлечения сущностей всё чаще применяются крупные языковые модели (Large Language Models, LLM). Такие модели способны с большой точностью извлекать из текста нужные сущности, при этом, не используя специальные правила. Однако их использование связано с рядом ограничений, среди которых можно отметить высокую вычислительную стоимость, зависимость качества работы от контекста и низкую интерпретируемость результатов. Кроме того, их применение в задачах промышленной обработки данных затруднено из-за требований к ресурсам и сложностью контроля качества.

2.4. Грамматические подходы и библиотека Yargy

Развитием rule-based методов являются подходы, основанные на использовании формальных грамматик. В таких методах структура сущности задаётся с помощью контекстно-свободных грамматик, позволяющие описывать сложные и вложенные конструкции. Грамматические методы отлично подходят для задачи извлечения адресной информации, где структура может быть формализована, но допускает множество вариантов записи. Кроме того, в отличие от методов на основе машинного обучения, такие подходы обеспечивают детерминированный и контролируемый процесс разбора текстов.

Одной из наиболее известных и открытых реализаций данного подхода для русского языка является библиотека Yargy [1]. В отличие от простых шаблонных методов, Yargy применяет контекстно-свободные грамматики и алгоритм Эрли [2] для синтаксического анализа, что позволяет учитывать вариативность структуры текста, которая встречается среди форм записи русских почтовых адресов. На рисунке 1 представлен пример извлечения из текста всех сущностей-регионов на основе ранее заданного правила с использованием библиотеки Yargy:

```
from yargy import Parser, rule, and_
from yargy.predicates import gram, is_capitalized, dictionary

GEO = rule(
    and_(
        gram('ADJF'), # так помечается прилагательное, остальные пометки описаны в
                      # http://pymorpho2.readthedocs.io/en/latest/user/grammemes.html
        is_capitalized()
    ),
    gram('ADJF').optional().repeatable(),
    dictionary({
        'федерация',
        'республика'
    })
)

parser = Parser(GEO)
text = '''
В Чеченской республике на день рождения ...
Донецкая народная республика провозгласила ...
Башня Федерация – одна из самых высоких ...
'''

for match in parser.findall(text):
    print([_.value for _ in match.tokens])
```

```
['Чеченской', 'республике']
['Донецкая', 'народная', 'республика']
```

Рисунок 1. Извлечение всех республик и федераций из текстов

К преимуществам данной библиотеки можно отнести:

- Возможность описать сложные и вложенные структуры через правила
- Эффективное извлечение структурированных имён и адресов
- Высокая интерпретируемость результата
- Эффективность при работе со сложной морфологией русского языка

Однако библиотека имеет некоторые недостатки:

- Ручное составление правил

Составление правил для разбора российских почтовых адресов является трудоёмкой задачей, поскольку необходимо учитывать большое количество вариантов записи, а также разнообразие географических объектов.

- Низкая производительность

В основе библиотеки лежит алгоритм Эрли, имеющий в худшем случае вычислительную сложность $O(n^3)$, где n — число токенов [2]. Дополнительно производительность ограничивается тем, что библиотека реализована на языке Python. В результате на тестовом наборе данных скорость обработки составляет порядка 15 адресов в секунду, что является недостаточным для задач промышленной обработки данных.

В связи с этим возникает необходимость разработки более производительного решения, сохраняющего преимущества грамматического подхода и удобного API библиотеки Yargy. В рамках данной работы предлагается создание библиотеки RENERT [5] на языке Rust, ориентированной на извлечение именованных сущностей из русскоязычных текстов. Предлагаемое решение направлено на сохранение точности и интерпретируемости исходного подхода при существенном увеличении скорости обработки текстов за счёт оптимизации алгоритмов и эффективного управления памятью.

3. Теоретическая основа и архитектура решения

3.1. Контекстно-свободная грамматика

Для описания структуры сложных текстовых сущностей, таких как почтовые адреса, могут применяться формальные языки и грамматики вместе с методами синтаксического анализа. В данной работе используется контекстно-свободная грамматика (КСГ), позволяющая задавать правила формирования последовательностей символов, определять вложенные структуры и выполнять синтаксический анализ за полиномиальное время.

В теории формальных языков и грамматики язык можно определить как множество цепочек, состоящих из конечного множества символов. Данные символы называются терминальными или терминалами, представляющие собой базовые элементы текста: слова, знаки препинания или токены. Помимо терминальных символов, в КСГ используются нетерминальные символы, которые описывают синтаксические категории и используются для задания структурных особенностей языка. В рамках использования КСГ в задаче обработки естественного языка нетерминалы можно интерпретировать как синтаксические классы, объединяющие группы конструкций, например, таких как «адрес», «улица» или «регион».

Контекстно-свободная грамматика задаётся конечным множеством правил переписывания продукций вида

$$A \rightarrow \alpha$$

где A — нетерминальный символ, α — последовательность (цепочка) терминальных или нетерминальных символов. Один из нетерминалов выбирается в качестве начального (корневого) символа грамматики и определяет весь язык.

Процесс получения из последовательности α за один шаг последовательности β с использованием одного из правил грамматики

принято обозначать как $\alpha \Rightarrow \beta$. Если цепочка β может быть получена за конечное число шагов, используется обозначение $\alpha \Rightarrow^* \beta$. Последовательность таких преобразований называется выводом (β из α).

Цепочки, которые могут быть получены из начального символа грамматики, называются сентиментальными формами. Цепочку, состоящую только из терминальных символов, называют предложением. Множество всех предложений, которые могут быть получены с помощью грамматик, образуют язык, задаваемый данной грамматикой.

Любую сентиментальную форму можно хотя бы одним способом представить в виде дерева разбора (или деревом вывода). Дерево разбора показывает, какие правила применялись при разборе, как формировалась итоговая последовательность, однако не фиксирует порядок их применения. Рассмотрим пример для грамматики AE , который имеет вывод

$$E \Rightarrow E + T \Rightarrow E + P \Rightarrow T + P \Rightarrow T * P + P.$$

Данный вывод грамматики можно представить в виде дерева разбора на рисунке 2:

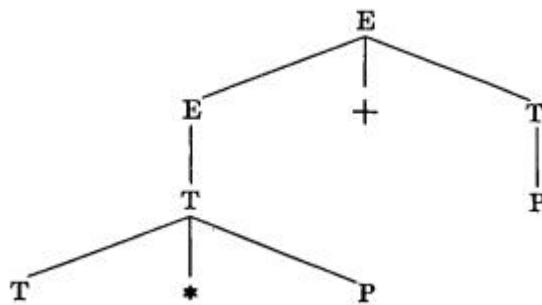


Рисунок 2. Вывод грамматики в виде дерева разбора [2]

Стоит отметить, что для одной и той же цепочки может существовать несколько различных ветвей разбора, что приводит к явлению неоднозначности грамматики. Грамматика называется однозначной, если для каждой цепочки существует единственное дерево разбора [2].

Для работы с грамматикой в КСГ существует два вида алгоритмов. Распознаватель (recognizer) определяет, принадлежит ли входная цепочка языку, заданному грамматикой. Анализатор (parser) решает более сложную задачу — проверяет корректность грамматики и строит множество всех допустимых деревьев вывода для выбранной цепочки [2].

Использование КСГ может быть актуальным в задаче извлечения структурированных сущностей, таких как почтовые адреса. Адрес можно рассматривать как последовательность компонентов (регион, город, улица, дом и т.д.), подчиняющуюся определённым правилам, но допускающую вариативность. Грамматический подход позволяет задавать правила для адресов и обрабатывать различные варианты записи.

3.2. Алгоритм Эрли

Алгоритм Эрли (Earley Parser) — это алгоритм синтаксического анализа, предназначенный для работы с контекстно-свободными грамматиками. Он был предложен Дж. Эрли в качестве способа определения принадлежности входной строки языку, заданному данной грамматикой. Алгоритм является универсальным алгоритмом разбора, поскольку может применяться к произвольным грамматикам и не требует их предварительного преобразования. Также алгоритм может выступать в роли и распознавателя, и анализатора.

Основная идея алгоритма заключается в последовательной обработке входной строки с построением набора промежуточных состояний. Для входной строки длины n формируется последовательность множеств состояний

$$S_0, S_1, \dots, S_n,$$

где каждое множество S_i описывает все возможные варианты частичного разбора на позиции i .

Каждое состояние представляет собой частично распознанное правило грамматики и задаётся как четвёрка [2]:

$$(p, j, f, \alpha),$$

где:

- p — номер правила;
- j — позиция точки внутри правила (степень распознавания);
- f — позиция во входной строке, с которой началось применение правила;
- α — строка просмотра вперёд (look-ahead длины k);

Таким образом, состояние одновременно хранит информацию о применяемом правиле, текущем этапе разбора и позиции во входной строке.

Работа алгоритма Эрли начинается с добавления начального состояния во множество S_0 . При этом вводится дополнительное правило [2]:

$$D_0 \rightarrow R \dashv ,$$

где R — начальный символ грамматики. Затем для каждой позиции входной строки $i = 0 \dots n$ выполняется три основные операции:

- **Predict**— добавляет правила для нетерминала, стоящего справа от точки;
- **Scan**— проверяет совпадение терминала со входным символом;
- **Complete**— обновляет состояния, которые ожидали данный нетерминал, если правило полностью разобрано;

Если в результате обработки некоторое множество S_{i+1} пусто, то входная строка отвергается, т.е. не принадлежит данной контекстно-свободной грамматике. В противном случае, если в конечном множестве S_n присутствует состояние, соответствующее завершению начального правила, строка принимается [2].

Алгоритм сочетает элементы нисходящего и восходящего синтаксического анализа, что позволяет эффективно обрабатывать грамматики с рекурсией и неоднозначностью, избегая избыточных вычислений.

С точки зрения вычислительной сложности алгоритм Эрли имеет следующие характеристики [2]:

- В общем случае — $O(n^3)$;
- Для однозначных грамматик — $O(n^2)$;
- Для класса грамматик с ограниченным размером множества состояний — $O(n)$;

Используемая память в алгоритме оценивается как $O(n^2)$, однако при построении всех возможных деревьев разбора и их хранении в памяти может достигать сложности $O(n^3)$ [2].

Практическим примером использования алгоритма Эрли может служить его реализация в библиотеке Yargy [1] в качестве основы синтаксического анализа контекстно-свободных грамматик. Это позволяет работать со сложными и неоднородными структурами русскоязычных почтовых адресов без дополнительного преобразования грамматики.

3.3. Архитектура библиотек Yargy и RENERT

Архитектура системы синтаксического анализатора в Yargy [1] построена как набор взаимосвязанных компонентов, выполняющих разные этапы обработки текста. Аналогичный подход был использован при разработанной библиотеке RENERT [5] с сохранением логики работы, структуры модулей и алгоритмической основы.

Обработка текста в представленных библиотеках идет последовательно и включает несколько этапов: токенизацию входной строки,

морфологический анализ токенов, применение грамматических правил и построение результата синтаксического разбора. Такое разделение упрощает сопровождение системы и делает архитектуру более гибкой.

В архитектуре библиотеки можно выделить несколько основных компонентов. Модуль Token используется для разбиения текста на токены и хранения информации о них. Морфологический анализ выполняется модулем Morph, который определяет грамматические характеристики слов и их нормальные формы. Описание контекстно-свободных грамматик вынесено в модуль Rule. Для проверки свойств токенов используются предикаты (Predicates), позволяющие задавать ограничения по словарям и грамматическим признакам. Синтаксический анализ реализуется модулем Parser, основанным на алгоритме Эрли. Дополнительно в библиотеке присутствует модуль Pipeline, предназначенный для построения сложных словарных конструкций и комбинированных правил.

Разработанная библиотека RENERT сохраняет аналогичное разделение модулей системы, как у оригинальной реализации Yargy. Такой подход позволяет:

1. Использовать аналогичную модель описание грамматик;
2. Обеспечивать близкий с Yargy API для пользователя;
3. Упростить перенос существующих правил и сценариев использования.

Оценивание сходства интерфейса и функциональности библиотек осуществлялся через визуальное оценивание API библиотек и реализации в RENERT аналогичных Yargy тестовых функций, которые должны на тех же данных давать такой же результат.

Однако, несмотря на архитектурное сходство, существуют ключевые различия между Yargy и RENERT, которые заключаются в особенностях реализаций, обусловленных выбором языка программирования. В

библиотеке Yargy используется язык программирования Python, который упрощает разработку и предоставляет удобный API пользователю, но накладывает ограничения на производительность, особенно в вычислительно сложных местах библиотеки, таких как синтаксический анализ или построение и обход графов правил разбора. В библиотеке RENERT реализация выполнена на языке Rust, предоставляющие в разработке такие преимущества как эффективное управление памятью, zero-copy обработки данных, высокая производительность и т.д.

4. Практическая реализация и оптимизация

Несмотря на универсальность алгоритма Эрли при работе с грамматикой, его высокая вычислительная сложность $O(n^3)$ в сочетании с интерпретируемой природой языка программирования Python и динамической типизацией переменных приводит к низкой производительности библиотеки Yargy при больших объёмах данных.

В связи с этим основная задача разработки RENERT заключается в сохранении архитектурных и алгоритмических преимуществ оригинальной библиотеки при существенном увеличении скорости обработки данных. Это достигается при помощи оптимизации реализации, выбора эффективных структур данных и использования возможностей языка Rust.

4.1. Выбор Rust

За многие годы использования в разработке языков высокого уровня они прошли долгий путь совершенствования и улучшений. Однако существует две проблемы, которые оказались трудно решаемые [6]:

1. **Безопасность кода.** При разработке программах на языках C и C++ становится проблематично управлять памятью и отслеживать неопределённое поведение программы. Это ведёт к брешам в системе и утечкам в памяти. Согласно проведённому в 2019 году исследованию компании Microsoft, около 70% всех уязвимостей безопасности были вызваны проблемами небезопасного использования памяти [7].
2. **Использование многопоточности.** Использование многопоточного кода и конкурентности может стать причиной появления новых классов ошибок, а также значительно усложняет воспроизведение и отладку уже известных проблем.

Чтобы решить перечисленные проблемы, компанией Mozilla был разработан компилируемый язык программирования Rust, имеющий такую же производительность, как у C и C++.

Помимо безопасной работы с памятью, язык Rust обладает рядом особенностей, которые делают его удобным для разработки высокопроизводительных библиотек обработки текстов, таких как RENERT. Одним из важных преимуществ Rust является высокая скорость выполнения программ, достигаемая за счёт компиляции в машинный код и отсутствия сборщика мусора в ядре языка [6]. Для задач синтаксического анализа это особенно важно, поскольку обработка текстов связана с большим объемом операций над строками, токенами и структурами разбора.

Дополнительным преимуществом Rust является поддержка zero-cost abstractions — механизмов абстракции, практически не создающих дополнительных накладных расходов во время выполнения программы. Благодаря этому удалось реализовать модульную архитектуру парсера и сохранить производительность даже при использовании сложных структур данных и контекстно-свободных грамматик.

Практическое значение для RENERT также имеет поддержка безопасной многопоточности. Возможность параллельной обработки данных позволяет эффективнее использовать многоядерные процессоры при анализе больших объёмов текстовой информации, а система владения и заимствования снижает вероятность ошибок, связанных с конкурентным доступом к памяти.

Стоит отметить, что развитая экосистема инструментов (Cargo, встроенная система тестирования, генерация документации) способствует ускорению разработки, сопровождения и верификации программного обеспечения, что является важным фактором при создании исследовательских и прикладных программ.

Таким образом, выбор Rust в качестве языка реализации RENERT обусловлен сочетанием высокой производительности, безопасности и удобства разработки, что делает этот язык программирования эффективной основой для построения промышленного синтаксического анализатора.

4.2. Программная реализация

В библиотеки RENERT основные типы и методы реэкспортируются из корневого модуля библиотеки `lib.rs` и представлены на Рисунок 3:

```
pub mod error;
pub(crate) mod internal;
pub mod interpretation;
pub mod morph;
pub mod parser;
pub mod pipeline;
pub mod predicates;
pub mod relations;
pub mod rule;
pub mod span;
pub mod token;
pub mod tree;

pub use parser::Parser;
pub use predicates::constructors::{and, eq, not, or};
pub use relations::{and_relation, not_relation, or_relation};
pub use rule::builder::eps as empty;
pub use rule::builder::{forward, main_term, or_, pred, rule, term, RuleBuilder, RuleId};
pub use rule::registry::RuleRegistry;
```

Рисунок 3. Реэкспорт ключевых структур и методов библиотеки RENERT

Такой набор реэкспортов формирует DSL-подобный интерфейс для описания грамматик, близкий по стилистике к Python-библиотеке Yargy, но с явным разделением одноимённых операций по уровням системы типов: *or_* — для правил, *or* — для предикатов, *or_relation* — для отношений согласования (и аналогично для *and* и *not*).

Описание контекстно-свободных грамматик строится через *RuleBuilder*, реализующий fluent-интерфейс. Поддерживаются конкатенация (через *rule([...])* или оператор *+*), альтернатива (*or_([...])* или оператор *|*), опциональные элементы (*.optional()*), повторения с произвольными или

ограниченными границами (*.repeatable()*, *.repeatable_with(min, max, ...)*), именованые (*.named("...")*) и рекурсивные правила через *forward()* с отложенным *define*. Терминалы грамматики выражаются через предикаты над токенами, предоставляя возможность задавать ограничения по строковому значению (*eq*, *caseless*, *normalized*, *dictionary*, *in_*), по числовым (*gte*, *lte*) и морфологическим признакам (*gram()*, *is_title()*, *is_upper()*, *is_digit()*, *length_eq()*). Например, проверка принадлежности токена существительного к титульному регистру с одновременным вхождением в словарь городов записывается одной композицией, представленной на Рисунок 4:

```
let city = and(vec![gram("NOUN"), is_title(), dictionary(&["москва", "казань"])]);
```

Рисунок 4. Пример построения правила на основе DSL-подобного интерфейса

Подобный подход позволяет декларативно описывать синтаксические конструкции и учитывать особенности русского языка — падежи, род и число — без ручной обработки морфологических форм.

Реестр *RuleRegistry* отвечает за хранение нормализованных правил, выдачу монотонных *RuleId* и индексацию всех вложенных подправил BFS-обходом графа. Метод *validate()* выполняет структурную проверку грамматики: ловит правила без продукций, пустые продукции и неопределённые forward-узлы, возвращая ошибки типа *RuleValidationError* ещё до запуска парсера.

Токенизация текста представлена в модуле **token**. Базовый класс *Tokenizer* использует регулярные выражения и выделяет фиксированный набор типов (Russian, Latin, Int, Punct, Email, Phone, Domain, EOL, Other). Морфологический токенизатор *MorphTokenizer* оборачивает базовый токенизатор и для русских слов добавляет полный набор морфологических разборов (*Vec<Form>*) на основе словаря *OpenCorpora*, формируя

унифицированное перечисление `AnyToken::{Plain, Morph}`. Сам словарь подготавливается однократным вызовом `renert::init(xml, cache_dir)` или загружается из ранее собранного кэша через `renert::load(dict_dir)`.

Центральным компонентом системы является структура *Parser*, реализующий алгоритм Эрли для произвольных контекстно-свободных грамматик. Анализ ведётся над структурой *Chart*, состоящей из колонок Earley-состояний, по одной на каждую позицию между токенами; в каждой колонке последовательно выполняются три классических шага алгоритма — *predict*, *scan* и *complete*, — с мемоизацией предсказаний и фильтрацией продукций по lookahead для ускорения разбора. Основная структура парсера хранит ссылку на реестр правил, идентификатор корневого правила, токенизатор и пользовательский тэггер.

Высокоуровневый API включает четыре семейства методов: *find()* и *findall()* — поиск первого и всех непересекающихся совпадений; *r#match()* — проверка, что корневое правило покрывает вход целиком; *extract()* — полный пайплайн с выдачей дерева разбора и привязки к тексту. Дополнительно пользователю доступны *find_text()* и *findall_text()*, возвращающие только значения токенов в виде `Vec<String>`, а также *parse(&[Token])* для работы с уже токенизированным входом — благодаря трейту *FindInput* методы *find()* и *findall()* полиморфны по типу входа (`&str`, `&[Token]`, `&Vec<Token>`).

Для типовых словарных задач предусмотрен модуль **pipeline** — газеттиры, заменяющие громоздкие конструкции из `or_` и `rule`. Функции *pipeline()*, *caseless_pipeline()* и *morph_pipeline()* принимают список фраз и автоматически собирают соответствующее правило: например, *morph_pipeline(["электронный дневник"])* распознает «электронным дневником», «электронные дневники» и другие словоформы благодаря морфологической нормализации по всем падежам и числам.

Дополнительно реализован механизм интерпретации совпадений в структурированные факты — модуль **interpretation**. Схема факта объявляется макросом *fact!()*, после чего фрагменты грамматики связываются с полями через *.interpretation(Fact::*field*)*, а корневое правило — через *.interpretation_fact::*<Fact>*()*. Также в этом модуле доступны методы нормализации значений: *.normalized()* для приведения к лемме, *.inflected(&["*nomn*", "*sing*"])* для приведения к согласованной форме, *.custom(*closure*)* для произвольного преобразования.

После успешного разбора метод *Match::*fact*(&*registry*)* возвращает типизированную запись со списком пар «имя поля + *FactValue*», именем факта и набором символьных диапазонов. Если пользователь хочет использовать эту запись для дальнейшей работы, то метод *to_json_value()* возвращает объект, пригодный для прямой сериализации в JSON, БД или внешний API.

Для обработки русскоязычных текстов в библиотеке также реализован механизм согласования словоформ. Модуль **relations** содержит методы, позволяющие проверять согласование слов по роду, числу и падежу. При разборе текста парсер исключает варианты, в которых соседние элементы грамматической конструкции не согласуются между собой по соответствующим морфологическим признакам.

Применение данного подхода организации библиотеки позволяет сохранить концептуальную и синтаксическую близость между API библиотек Yargy и RENERT. В результате описание грамматик и правил в обеих системах выполняется схожим образом. На Рисунок 5 представлен код на основе библиотеки RENERT, аналогичный по функциональности коду из рисунка 1:

```

use yargy_core::predicates::{and, dictionary, gram, is_title};
use yargy_core::{pred, Parser, RuleRegistry};

let text = r#"
    В Чеченской республике на день рождения ...
    Донецкая народная республика провозгласила ...
    Башня Федерация — одна из самых высоких ...
"#;

let geo_rule = (
    pred(and(vec![
        gram("ADJF"),
        is_title(), // аналог is_capitalized()
    ]))
    + pred(gram("ADJF").optional().repeatable())
    + pred(dictionary(&["федерация", "республика"]))
).build(());

let mut registry = RuleRegistry::new();
let geo_id = registry.add(geo_rule);

let parser = Parser::new(&registry, geo_id);

for m in parser.findall(text) {
    let tokens = m.tokens();
    let values: Vec<&str> = tokens.iter().map(|t| t.value.as_ref()).collect();
    println!("{:?}", values);
}
// Сразу Vec<String> на каждый матч:
for values in parser.findall_text(text) {
    println!("{:?}", values);
}

```

Output:

```

["Чеченской", "республике"]
["Донецкая", "народная", "республика"]

```

Рисунок 5. Извлечение всех республик и федераций из текстов с помощью библиотеки RENERT

Можно заметить, что интерфейс RENERT не сильно различается от API Yargy, чем и являлось одной из приоритетной задачей при реализации библиотеки.

4.3. Оптимизация в библиотеке RENERT

Производительность библиотеки RENERT обеспечивается совокупностью алгоритмических, архитектурных и низкоуровневых оптимизаций, направленных на сокращение количества состояний парсера,

уменьшение числа операций копирования данных и повторного выполнения вычислений. Основное внимание при разработке уделялось оптимизации алгоритма Эрли, морфологического анализа и работы с памятью, поскольку именно эти компоненты формируют основную вычислительную нагрузку при обработке русскоязычных текстов.

Одной из ключевых проблем классического алгоритма Эрли является быстрый рост количества состояний на этапе **predict**, при котором для нетерминала раскрываются все возможные продукции грамматики. При использовании сложных и рекурсивных грамматик это приводит к существенному увеличению объёма вычислений и потребления памяти. Для уменьшения числа создаваемых состояний в RENERT применяется lookahead-фильтрация продукций. Перед добавлением нового состояния система проверяет, может ли соответствующая продукция начинаться с текущего токена входной последовательности. Продукции, заведомо не подходящие для дальнейшего разбора, отбрасываются ещё до создания новых состояний парсера. В тоже время механизм мемоизации предсказаний предотвращает повторное раскрытие одинаковых правил в пределах одной колонки в таблице состояний. В результате существенно сокращается количество промежуточных состояний при работе с грамматиками, содержащие общие подправила и рекурсивные конструкции.

Для ускорения этапа **complete** используется индексирование родительских состояний, благодаря чему завершённый нетерминал обновляет только связанные с ним состояния, а не всю колонку Chart целиком. Это помогает уменьшить количество линейных проходов по структурам данных и уменьшает асимптотические затраты на завершение разбора.

Также в библиотеке реализована система дедупликации состояний парсера. Каждая колонка Chart хранит хеш-индекс уже существующих

состояний, что позволяет быстро обнаруживать и исключать дубликаты. Проверка выполняется в два этапа: сначала сравниваются хеш-значения, после чего при необходимости выполняется точная проверка эквивалентности состояний. Кроме этого применяется механизм совместного использования поддеревьев разбора. Уже построенные фрагменты дерева могут повторно использоваться в разных вариантах разбора без создания новых копий.

Морфологический анализ текста оптимизирован за счёт использования глобального экземпляра *MorphTokenizer*, который разделяется между всеми парсерами. Инициализация токенизатора выполняется только при первом обращении к морфологическому анализу. Дополнительно применяется LRU-кеш для хранения результатов разбора часто встречающихся словоформ. Поскольку многие слова в тексте повторяются, это сокращает количество повторных обращений к словарю, и ускорить обработку данных.

Для эффективной работы с памятью, в библиотеке RENERT большинство сложных структур данных, включая правила грамматики, деревья разбора и токены, используют механизм совместного владения через Arc. Это даёт возможность нескольким состояниям парсера вновь использовать одни и те же объекты без их копирования. При возврате нескольких совпадений исходный текст и массив токенов создаются только один раз и затем разделяются между всеми объектами результата.

В результате последовательные оптимизации позволяют повысить производительность библиотеки RENERT при обработке русскоязычных текстов и сложных контекстно-свободных грамматик. Алгоритмические улучшения уменьшают количество циклов парсера Эрли, кеширование малых чисел повторных морфологических анализов, оптимизация работы с памятью и сокращение расходов при построении неоднозначных деревьев. Благодаря использованию языка Rust и эффективному управлению этой

библиотеки обеспечивается производительность, близкая к системным решениям на С и С++, сохраняя при этом высокий уровень безопасности памяти и удобство разработки.

5. Вычислительный эксперимент

5.1 Описание эксперимента

Для оценки производительности разработанной библиотеки RENERT и её сравнения с библиотекой Yargy был проведён эксперимент на задаче распознавания русскоязычных почтовых адресов. В качестве тестовых данных использовались адреса из открытого набора данных Russian Houses [8]. На основе исходного набора были сформированы три датасета различного размера, содержащие 500, 2000 и 5000 адресов соответственно.

В ходе эксперимента библиотеки RENERT и Yargy выполняли разбор адресов при помощи одинакового набора контекстно-свободных грамматических правил. Грамматика описывала основные компоненты российских адресов: регион, населённый пункт, улицу, номер дома, корпус и квартиру. Пример разбора адреса по составленным для эксперимента правилам библиотекой RENERT представлен на рисунке 6:

```
Россия, обл. Московская, г. Ступино, рп Михнево, ул. Советская, д. 31а
Strana(
  name='Россия'
)
Oblast(
  name='Московская',
  kind='обл.'
)
Gorod(
  name='Ступино',
  kind='г.'
)
Gorod(
  name='Михнево',
  kind='рп'
)
Ulitsa(
  name='Советская',
  kind='ул.'
)
Dom(
  number='31а',
  kind='д.'
)
```

Рисунок 6. Извлечение всех республик и федераций из текстов с помощью библиотеки RENERT

Для измерения производительности использовались benchmark-тесты, размещённые в модуле `/benches` библиотеки RENERT. Измерялось полное время обработки каждого датасета, включая токенизацию и выполнение синтаксического анализа. Для уменьшения влияния случайных факторов каждый эксперимент выполнялся несколько раз, после чего вычислялось среднее значение времени выполнения. Библиотека RENERT запускалась в release-сборке Rust, а библиотека Yargy — в среде Python. Основной целью эксперимента являлась оценка влияния архитектурных и алгоритмических оптимизаций RENERT на скорость выполнения грамматического анализа.

5.2. Результаты эксперимента

В результате выполнения benchmark-тестов были получены данные о времени обработки трёх наборов адресов библиотеками RENERT и Yargy. Результаты представлены в таблице 1.

Dataset	Number of addresses	RENERT	Yargy	SpeedUp (Python/Rust)
address_small	500	1.630 s	32.066 s	19.68x
address_medium	2000	6.576 s	140.418 s	21.35x
address	5000	16.542 s	387.910 s	23.45x

Таблица 1. Результаты эксперимента на трёх наборах данных

5.3. Анализ полученных результатов

Результаты вычислительного эксперимента подтверждают высокую эффективность разработанной библиотеки RENERT при решении задачи грамматического разбора русскоязычных адресов. В среднем библиотека выполняла обработку данных более чем в 21 раз быстрее по сравнению с Yargy. Средняя скорость разбора RENERT составляла 310 адресов в секунду, в то время как у Yargy скорость составляла всего 15 адресов в секунду.

Сочетание преимуществ компилируемого языка Rust и оптимизаций в алгоритме Эрли являются основными причинами полученного ускорения.

Более подробно ознакомиться с результатами benchmark-тестов, кодом парсеров и наборов контекстно-свободных грамматических правил для разбора адресов можно в исходном коде библиотеки RENERT в модуле /benches.

Таким образом, результаты вычислительного эксперимента показывают, что библиотека RENERT сохраняет возможности грамматического анализа библиотеки Yargy, одновременно обеспечивая значительный прирост производительности.

6. Заключение

Задача извлечения именованных сущностей из русскоязычных текстов часто возникает в прикладных системах, связанной с автоматизацией документооборота, логистикой, CRM-системах и хранении пользовательских данных. Особенно часто представленные методы используются при обработке почтовых адресов. Однако автоматический разбор русскоязычных почтовых адресов осложняется особенностями языка: слова изменяются по падежам и числам, а сами адреса нередко записываются в разных форматах, содержат сокращения или пропуски отдельных элементов. Следовательно, задача обработки адресной информации требует более гибких методов синтаксического анализа.

В ходе работы были изучены существующие подходы для извлечения именованных сущностей, методы грамматического анализа текста, а также архитектурные особенности библиотеки Yargy. По результатам анализа была разработана библиотека RENERT на языке программирования Rust, использующая грамматический подход к разбору текстов и сохраняющая совместимый с Yargy подход к описанию правил и использованию парсера.

В процессе разработки были реализованы основные компоненты системы синтаксического анализа, основанной на алгоритме Эрли, а также предложены оптимизации, направленные на повышение производительности парсера. Среди них можно выделить lookahead-фильтрацию правил, мемоизацию predict-операций и предварительную индексацию грамматических конструкций.

Для оценки эффективности разработанного решения был проведён вычислительный эксперимент на задаче разбора русскоязычных почтовых адресов. Результаты эксперимента показали, что библиотека RENERT обеспечивает значительное повышение производительности по сравнению с библиотекой Yargy. В среднем ускорение составило более 21 раза при обработке наборов данных различного объёма.

Таким образом, поставленная цель работы была достигнута: разработана высокопроизводительная библиотека для грамматического анализа русскоязычных текстов, сочетающая возможности контекстно-свободных грамматик с преимуществами языка Rust. Полученные результаты подтверждают перспективность использования Rust для построения производительных систем извлечения именованных сущностей и дальнейшего развития грамматических методов обработки естественного языка.

Список литературы

1. Yargy: библиотека для извлечения сущностей из текста [Электронный ресурс]. — URL: <https://github.com/natasha/yargy/tree/master/yargy>
2. Jay Earley. 1970. An efficient context-free parsing algorithm. Commun. ACM 13, 2 (Feb 1970), 94–102. <https://doi.org/10.1145/362007.362035>
3. Бручес, Елена Павловна. Методы и алгоритмы распознавания и связывания сущностей для построения систем автоматического извлечения информации из научных текстов: автореферат дис. ... кандидата технических наук : 05.13.17 / Бручес Елена Павловна; [Место защиты: ФГБОУ ВО «Сибирский государственный университет телекоммуникаций и информатики»]. — Новосибирск, 2021. — 18 с..https://www.iis.nsk.su/files/abstract/disser/dissertaciya_bruches_final.pdf
4. Правила оказания услуг почтовой связи: утв. приказом Минцифры России от 17.04.2023 № 382 (ред. действующая). — Текст: электронный // [Официальный интернет-портал правовой информации](http://publication.pravo.gov.ru/document/0001202306020025). — URL: <http://publication.pravo.gov.ru/document/0001202306020025>
5. RENERT: Rust-библиотека для извлечения структурированной информации из русскоязычных текстов. [Электронный ресурс]. — URL:<https://github.com/media-tel/renert>
6. Блэнди Дж., Орендорф Дж. Программирование на языке Rust = Programming Rust. — [ДМК Пресс](#), 2018. — 550 с. — [ISBN 978-5-97060-236-2](#).
7. ["Microsoft: 70 percent of all security bugs are memory safety issues"](#). ZDNET. Retrieved 21 September 2022.
8. Russian houses [Электронный ресурс] / Gasfarmuhametdinov // Kaggle: [сайт] URL: <https://www.kaggle.com/datasets/gasfarmuhametdinov/russian-houses> (дата обращения: 11.05.2026).